

```
/*
Purpose:
-----
This script calculates the Addition, Subtraction, Multiplication and
Division of two numbers.
```

```
*/
```

```
Declare    @Number1 int = 50,
           @Number2 int = 10,
           @Addition int,
           @Subtraction int,
           @Multiplication int,
           @Division Decimal(10, 2);

set @Addition = @Number1 + @Number2;
set @Subtraction = @Number1 - @Number2;
set @Multiplication = @Number1 * @Number2;

if @Number2 <> 0
    set @Division = @Number1 / @Number2;
else
    set @Division = NULL;

print 'Addition = ' + CAST(@Addition as nvarchar(10));
print 'Subtraction = ' + CAST(@Subtraction as nvarchar(10));
print 'Multiplication = ' + CAST(@Multiplication as nvarchar(10));
print 'Division = ' + ISNULL(CAST(@Division as nvarchar(50)),
'Cannot divide by zero');
```

```
----- Query
```

```
select
    @Number1 + @Number2 AS Addition,
    @Number1 - @Number2 AS Subtraction,
    @Number1 * @Number2 AS Multiplication,
```

```
CAST(@Number1 AS DECIMAL(10,2)) / NULLIF(@Number2, 0) AS Division;
```

----- Output

Results		Messages		
	Addition	Subtraction	Multiplication	Division
1	60	40	500	5.000000000000000

```
/*
```

```
Purpose:
```

```
-----
```

```
This script Swap 2 numbers with/without using 3rd Variable.
```

```
*/
```

```
Declare @a int = 1,
```

```
        @b int = 2,
```

```
        @temp int;
```

```
        print 'Before Swapping: ' + cast(@a as varchar(5)) + ' ' +  
cast(@b as varchar(5));
```

```
        set @temp = @a;
```

```
        set @a = @b;
```

```
        set @b = @temp;
```

```
        print 'After Swapping: ' + cast(@a as varchar(5)) + ' ' +  
cast(@b as varchar(5));
```

----- Without using 3rd Variable

```
Declare @a1 int = 1,
```

```
        @b1 int = 2,
```

```
        @temp1 int;
```

```
        print 'Before Swapping without using 3rd variable: ' +  
cast(@a1 as varchar(5)) + ' ' + cast(@b1 as varchar(5));
```

```

set @a1 = @a1 + @b1;

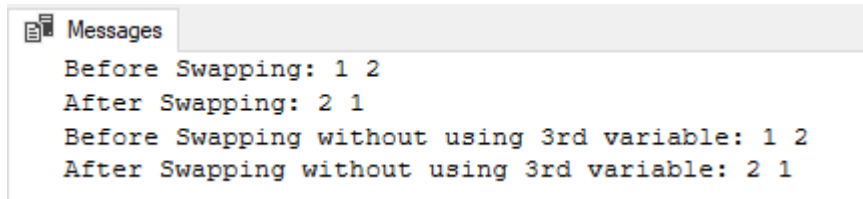
set @b1 = @a1 - @b1;

set @a1 = @a1 - @b1;

print 'After Swapping without using 3rd variable: ' + cast(@a1
as varchar(5)) + ' ' + cast(@b1 as varchar(5));

```

----- Output



Messages

```

Before Swapping: 1 2
After Swapping: 2 1
Before Swapping without using 3rd variable: 1 2
After Swapping without using 3rd variable: 2 1

```

```

/*
Purpose:
-----
This script find Number is Even or Odd.
*/

```

```

Declare @Number int = 7;

if (@Number % 2 = 0)
begin
    print 'Even Number: ' + cast(@Number as nvarchar(20))
end

else
begin
    print 'Odd Number: ' + cast(@Number as nvarchar(20))
end

```

----- Query

```

select case

    when (@Number % 2 = 0)

    then 'Even Number'

```

```

        else 'Odd Number'
        end as Message,

@Number as Number;

```

----- Output

	Message	Number
1	Even Number	24

	Message	Number
1	Odd Number	21

```

/*
Purpose:
-----
This script find year is Leap year or not.
*/

```

```

Declare @Year int = 2005;

    if (@Year % 4 = 0)
    begin
        print 'Leap Year: ' + cast(@Year as nvarchar(20))
    end

    else
    begin
        print 'Not Leap Year: ' + cast(@Year as nvarchar(20))
    end

```

----- Query

```

select case

        when (@Year % 4 = 0)
        then 'Leap Year'
        else 'Not Leap Year'
        end as Message,

@Year as Year;

```

----- Output

Results		Messages	
	Message	Year	
1	Leap Year	2000	

Results		Messages	
	Message	Year	
1	Not Leap Year	2005	

/*

Purpose:

This script calculates the factorial of a given number.

Logic:

1. Validates whether the input number is negative or not and Returns an appropriate message.
3. Handles the special case of $0! = 1$.
4. Uses a WHILE loop to calculate the factorial for positive numbers.
5. Displays the calculated factorial value.

*/

```
Declare @n int = 0,
        @Factorial bigint = 1,
        @i int = 1;

if (@n < 0)
begin
    print 'Factorial is not defined for negative numbers.';
end
else if ( @n = 0)
begin
    print 'Factorial = 1';
end
else
begin
    while (@i <= @n)
```

```

begin
    set @Factorial = @Factorial * @i;
    set @i = @i + 1;
end

print 'Factorial = ' + cast(@Factorial as nvarchar(10));
end

```

----- Output

Results		Messages	
	Factorial		
1	120		

/*

Purpose:

This script checks whether the given input is a palindrome.

Logic:

1. Accept input as NVARCHAR.
2. Reverse the input using REVERSE().
3. Compare the original and reversed values.
4. Display whether the input is a palindrome or not.

Examples:

121 -> Palindrome
 123 -> Not Palindrome
 eye -> Palindrome
 madam -> Palindrome
 hello -> Not Palindrome
 */

```

Declare @Input nvarchar(10) = '121',
        @Reversed nvarchar(10);

set @Reversed = REVERSE(@Input);

if @Input = @Reversed
begin

```

```

        print 'Palindrom = ' + @Input;
    end
    else
    begin
        print 'Not Palindrom = ' + @Input;
    end

```

----- Query

```

select  @Input as Input,
        @Reversed as Output,
        case
            when @Input = @Reversed
                then 'Palindrome'
            else
                'Not Palindrome'
        end as Result;

```

----- Output

	Input	Output	Result
1	121	121	Palindrome

	Input	Output	Result
1	1213	3121	Not Palindrome

/*

Purpose:

This script calculates the power of a number using a WHILE loop.

Formula:

$x^y = x \times x \times x \dots (y \text{ times})$

Examples:

$$2^3 = 8$$

$$5^2 = 25$$

$$10^0 = 1$$

Logic:

1. Accept the base number (@x).
2. Accept the exponent value (@y).
3. Initialize the result variable to 1.
4. Multiply the result by the base number repeatedly until the exponent count is reached.
5. Display the calculated power value.

Input:

@x : Base number

@y : Exponent value

*/

```
Declare @x int = 2,
        @y int = 3,
        @i int = 1,
        @p int = 1;

while ( @i <= @y )
begin
    set @p = @p * @x;
    set @i = @i + 1;
end
```

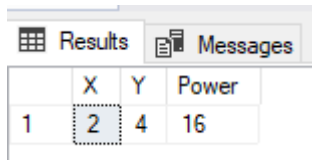


```
end  
print 'Power = ' + cast( @p as nvarchar(10) );
```

----- Query

```
select @x as x, @y as y, @p as Power;
```

----- Output



	X	Y	Power
1	2	4	16

/*

Purpose:

This script calculates the Sum of a digit using a WHILE loop.

Examples:

123 = 1 + 2 + 3 = 6

Logic:

1. Accept the Number (@n).
2. Display the calculated Sum of value.

Input:

@n : Number

*/

```
Declare @n int = 13,  
        @Sum int = 0,  
        @k int = 0;
```

```

while ( @n > 0)
begin
    set @k = @n % 10;
    set @Sum = @Sum + @k;
    set @n = @n / 10;
end

print 'Sum Of Digite = ' + cast ( @Sum as nvarchar(10) );

```

----- Query

```
select @Sum as [Sum Of Digite];
```

----- Output

Results		Messages
	Number	Sum Of Digite
1	13	4

/*

Purpose:

This script checks whether a given number is an Armstrong Number.

Examples:

$$153 = 1^3 + 5^3 + 3^3 = 153$$

$$370 = 3^3 + 7^3 + 0^3 = 370$$

$$371 = 3^3 + 7^3 + 1^3 = 371$$

$$407 = 4^3 + 0^3 + 7^3 = 407$$

Logic:

1. Store the original number.
2. Extract each digit using the modulus (%) operator.
3. Calculate the cube of each digit.

4. Add the cubes to a running total.
5. Remove the processed digit using integer division.
6. Compare the calculated sum with the original number.
7. Display whether the number is an Armstrong Number.

Input:

@n : Number to be checked.

*/

```
Declare @n int = 153,
        @Sum int = 0,
        @k int = 1,
        @p int;

set @p = @n;

while ( @n > 0 )
begin
    set @k = @n % 10;
    set @k = @k * @k * @k;
    set @Sum = @Sum + @k;
    set @n = @n / 10;
end
```

----- Query

```
select @Sum as [Number],
       case
           when ( @Sum = @p )
               then 'Armstrong Number!'
           else
               'Not Armstrong Number!'
```

```
end as Message;
```

----- Output

Results		
Messages		
	Number	Message
1	153	Armstrong Number!

Results		
Messages		
	Number	Message
1	24	Not Armstrong Number!

```
/*
```

Purpose:

This script checks whether a given number is a Perfect Number.

Examples:

$6 = 1 + 2 + 3 = 6$

$28 = 1 + 2 + 4 + 7 + 14 = 28$

Logic:

1. Accept the input number.
2. Find all divisors of the number excluding itself.
3. Add all the divisors to calculate their sum.
4. Compare the sum of divisors with the original number.
5. Display whether the number is a Perfect Number or not.

Input:

@n : Number to be checked.

```
*/
```

```
Declare @n int = 6,  
        @Sum int = 0,  
        @k int = 1,
```

```

    @i int = 1;

    while ( @i < @n)
    begin
        set @k = @n % @i;

        if ( @k = 0 )
        begin
            set @Sum = @Sum + @i;
        end
        set @i = @i + 1;
    end
end

```

----- Query

```

select @Sum as [Number],
       case
           when ( @Sum = @n )
               then 'Perfect Number!'
           else
               'Not Perfect Number!'
       end as Message;

```

----- Output

Results		Messages
	Number	Message
1	6	Perfect Number!

Results		Messages
	Number	Message
1	1	Not Perfect Number!

/*

Purpose:

This script checks whether a given number is a Prime Number.

Definition:

A Prime Number is a number greater than 1 that has exactly two factors: 1 and itself.

Examples:

2, 3, 5, 7, 11, 13

Logic:

1. Accept the input number.
2. Find the count of factors.
3. If the number has exactly two factors, it is Prime.
4. Otherwise, it is Not Prime.

Input:

@n : Number to be checked.

*/

```
Declare @n int = 15,
        @Sum int = 0,
        @i int = 1,
        @Factors nvarchar(10) = '';

while ( @i <= @n)
begin
    if ( @n % @i = 0 )
    begin
```

```

                set @Factors = @Factors + cast(@i as
nvarchar(10)) + ', ';
                set @Sum = @Sum + 1;
            end
            set @i = @i + 1;
        end

```

----- Query

```

        select @n as [Number], @Factors as Factors, @Sum as
[Factor Counter],
        case
            when ( @Sum = 2 )
                then 'Prime Number!'
            else
                'Not Prime Number!'
            end as Message;

```

----- Output

Results		Messages		
	Number	Factors	Factor Counter	Message
1	7	1, 7,	2	Prime Number!

Results		Messages		
	Number	Factors	Factor Counter	Message
1	4	1, 2, 4,	3	Not Prime Number!

/*

Purpose:

This script generates the Fibonacci Series up to a specified number of terms.

Examples:

0, 1, 1, 2, 3, 5, 8, 13, 21...

Logic:

1. Initialize the first two numbers.
2. Calculate the next number as the sum of the previous two.
3. Repeat until the required number of terms is generated.

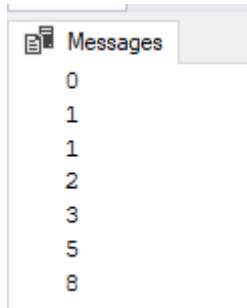
*/

```
Declare @n int = 7,  
        @i int = 1,  
        @t1 int = 0,  
        @t2 int = 1,  
        @nextTerm int;  
  
while ( @i <= @n )  
begin  
    if ( @i = 1 )  
    begin  
        print @t1;  
    end  
  
    else if ( @i = 2 )  
    begin  
        print @t2;  
    end  
  
    else  
    begin  
        set @nextTerm = @t1 + @t2;  
        print @nextTerm;  
        set @t1 = @t2;  
        set @t2 = @nextTerm;  
    end  
end
```



```
        set @i = @i + 1;
    end
```

----- Output



/*

Purpose:

This script counts the total number of digits in a given number.

Examples:

12345 -> 5

100 -> 3

7 -> 1

Logic:

1. Divide the number by 10 repeatedly.
2. Increment the counter for each iteration.
3. Continue until the number becomes zero.

*/

```
Declare @n int = 12345;
Declare @originalN int = @n;
Declare @count int = 0;
```

```
-- Handle 0 explicitly if necessary, otherwise the loop won't
execute
if @n = 0
    set @count = 1;
else
begin
    -- Ensure we work with absolute value to handle negative numbers
    set @n = ABS(@n);

    while @n > 0
    begin
        set @n = @n / 10;
        set @count = @count + 1;
    end
end
```

----- Query

```
select
    @originalN as InputNumber,
    @count as TotalDigits;
```

----- Output

Results		Messages
	InputNumber	TotalDigits
1	12345	5

/*

Purpose:

This script checks whether a given number is a Strong Number.

Examples:

$$\begin{aligned} 145 &= 1! + 4! + 5! \\ &= 1 + 24 + 120 \\ &= 145 \end{aligned}$$

Logic:

1. Extract each digit from the number.
2. Calculate the factorial of each digit.
3. Add all factorial values.
4. Compare the sum with the original number.
5. Display whether the number is Strong or Not Strong.

*/

```
Declare @n int = 145;
Declare @sum int = 0;
Declare @digit int;
Declare @fact int;
Declare @i int;
Declare @temp int = @n;

while @temp > 0
begin
    set @digit = @temp % 10;

    set @fact = 1;
    set @i = 1;

    while @i <= @digit
    begin
```

```

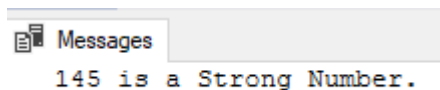
        set @fact = @fact * @i;
        set @i = @i + 1;
    end

    set @sum = @sum + @fact;
    set @temp = @temp / 10;
end

if @sum = @n
    print CAST(@n as nvarchar(10)) + ' is a Strong Number.';
else
    print CAST(@n as nvarchar(10)) + ' is not a Strong Number.';

```

----- Output



Messages
145 is a Strong Number.

```

/*
Purpose:
-----
This script calculates the total number of Vowels and Consonants
into string.
*/

```

```

DECLARE @String VARCHAR(100) = 'HELLO WORLD';
DECLARE @Index INT = 1;
DECLARE @Length INT = LEN(@String);
DECLARE @Char CHAR(1);

DECLARE @Vowels VARCHAR(100) = '';
DECLARE @Consonants VARCHAR(100) = '';

DECLARE @VowelCount INT = 0;
DECLARE @ConsonantCount INT = 0;

```

```

WHILE @Index <= @Length
BEGIN
    SET @Char = UPPER(SUBSTRING(@String, @Index, 1));

    IF @Char LIKE '[A-Z]'
    BEGIN
        IF @Char IN ('A', 'E', 'I', 'O', 'U')
        BEGIN
            SET @Vowels = @Vowels + @Char + ',';
            SET @VowelCount = @VowelCount + 1;
        END
        ELSE
        BEGIN
            SET @Consonants = @Consonants + @Char + ',';
            SET @ConsonantCount = @ConsonantCount + 1;
        END
    END

    SET @Index = @Index + 1;
END

-- Remove trailing comma
SET @Vowels = LEFT(@Vowels, LEN(@Vowels) - 1);
SET @Consonants = LEFT(@Consonants, LEN(@Consonants) - 1);

PRINT 'Vowels = ' + @Vowels;
PRINT 'Vowel Count = ' + CAST(@VowelCount AS VARCHAR(10));

PRINT 'Consonants = ' + @Consonants;

```

```
PRINT 'Consonant Count = ' + CAST(@ConsonantCount AS
VARCHAR(10));
```

----- Query

```
SELECT

    @Vowels AS Vowels,

    @VowelCount AS VowelCount,

    @Consonants AS Consonants,

    @ConsonantCount AS ConsonantCount;
```

----- Output

Results		Messages		
	Vowels	VowelCount	Consonants	ConsonantCount
1	E,O,O	3	H,L,L,W,R,L,D	7

```
/*
```

Purpose:

This script helps to Create Different * patterns. Used Case statement into it. User need to add just pattern name and In output it will automatically generate.

```
*/
```

```
DECLARE @Rows INT = 5;
DECLARE @PatternType VARCHAR(30) = 'Hollow Rhombus';
DECLARE @i INT = 1;
DECLARE @j INT = 1;
DECLARE @Line NVARCHAR(100) = '';

PRINT '--- Generating Pattern: ' + @PatternType + ' ---';
IF @PatternType = 'Right Half Pyramid'
BEGIN
    WHILE (@i <= @Rows)
    BEGIN
```

```

        WHILE (@j <= @i)
        BEGIN
            SET @Line = @Line + '* ';
            SET @j = @j + 1;
        END

        PRINT @Line;
        SET @i = @i + 1;
    END
END

ELSE IF @PatternType = 'Left Half Pyramid'
BEGIN
    WHILE (@i <= @Rows)
    BEGIN
        SET @Line = '';
        SET @j = 1;

        WHILE (@j <= (@Rows - @i))
        BEGIN
            SET @Line = @Line + '  ';
            SET @j = @j + 1;
        END

        SET @j = 1;
        WHILE (@j <= @i)
        BEGIN
            SET @Line = @Line + '* ';
            SET @j = @j + 1;
        END
    END

```

```

        PRINT @Line;
        SET @i = @i + 1;
    END
END

ELSE IF @PatternType = 'Full Pyramid'
BEGIN
    WHILE (@i <= @Rows)
    BEGIN
        SET @Line = '';

        SET @j = 1;
        WHILE (@j <= (@Rows - @i))
        BEGIN
            SET @Line = @Line + ' ';
            SET @j = @j + 1;
        END

        SET @j = 1;
        WHILE (@j <= (2 * @i - 1))
        BEGIN
            SET @Line = @Line + '*';
            SET @j = @j + 1;
        END

        PRINT @Line;
        SET @i = @i + 1;
    END
END

```



```

ELSE IF @PatternType = 'Hollow Full Pyramid'
BEGIN
    SET @i = 1;
    WHILE @i <= @Rows
    BEGIN
        SET @Line = SPACE(@Rows - @i);
        SET @j = 1;

        WHILE @j <= (2 * @i - 1)
        BEGIN
            IF @j = 1 OR @j = (2 * @i - 1) OR @i = @Rows
                SET @Line = @Line + '*';
            ELSE
                SET @Line = @Line + ' ';
            SET @j = @j + 1;
        END

        PRINT @Line;
        SET @i = @i + 1;
    END
END

ELSE IF @PatternType = 'Diamond Pyramid'
BEGIN
    SET @i = 1;
    WHILE (@i <= @Rows)
    BEGIN
        SET @Line = SPACE(@Rows - @i) + REPLICATE('*', @i);
        PRINT @Line;
    END
END

```

```

        SET @i = @i + 1;

    END

    SET @i = @Rows - 1;
    WHILE (@i >= 1)
    BEGIN
        SET @Line = SPACE(@Rows - @i) + REPLICATE('*', @i);
        PRINT @Line;
        SET @i = @i - 1;
    END
END

ELSE IF @PatternType = 'Hollow Diamond Pyramid'
BEGIN
    -- Upper half
    SET @i = 1;
    WHILE @i <= @Rows
    BEGIN
        SET @Line = SPACE(@Rows - @i);
        SET @j = 1;

        WHILE @j <= (2 * @i - 1)
        BEGIN
            IF @j = 1 OR @j = (2 * @i - 1)
                SET @Line = @Line + '*';
            ELSE
                SET @Line = @Line + ' ';
            SET @j = @j + 1;
        END

        PRINT @Line;
    END
END

```

```

        SET @i = @i + 1;
    END

    -- Lower half
    SET @i = @Rows - 1;
    WHILE @i >= 1
    BEGIN
        SET @Line = SPACE(@Rows - @i);
        SET @j = 1;

        WHILE @j <= (2 * @i - 1)
        BEGIN
            IF @j = 1 OR @j = (2 * @i - 1)
                SET @Line = @Line + '*';
            ELSE
                SET @Line = @Line + ' ';
            SET @j = @j + 1;
        END
        PRINT @Line;
        SET @i = @i - 1;
    END
END

ELSE IF @PatternType = 'Hollow Square'
BEGIN
    WHILE (@i <= @Rows)
    BEGIN
        SET @j = 1;
        SET @Line = '';
        WHILE (@j <= @Rows)

```

```

        BEGIN

            IF (@i = 1 OR @i = @Rows OR @j = 1 OR @j = @Rows)
                SET @Line = @Line + '* ';
            ELSE
                SET @Line = @Line + '  ';
            SET @j = @j + 1;
        END

        PRINT @Line;
        SET @i = @i + 1;
    END
END

ELSE IF @PatternType = 'Rhombus'
BEGIN
    WHILE (@i <= @Rows)
    BEGIN

        SET @Line = SPACE(@Rows - @i);

        SET @Line = @Line + REPLICATE('*', @Rows);

        PRINT @Line;
        SET @i = @i + 1;
    END
END

ELSE IF @PatternType = 'Hollow Rhombus'
BEGIN
    SET @i = 1;

```

```

WHILE @i <= @Rows
BEGIN
    -- Leading spaces to shift the rhombus
    SET @Line = SPACE(@Rows - @i);
    SET @j = 1;

    WHILE @j <= @Rows
    BEGIN
        -- Print '*' only at borders (first/last row or
first/last column)
        IF @i = 1 OR @i = @Rows OR @j = 1 OR @j = @Rows
            SET @Line = @Line + '*';
        ELSE
            SET @Line = @Line + ' ';
        SET @j = @j + 1;
    END

    PRINT @Line;
    SET @i = @i + 1;
END

END

ELSE IF @PatternType = 'Hour Glass'
BEGIN
    SET @i = @Rows;
    WHILE (@i >= 1)
    BEGIN
        SET @Line = SPACE(@Rows - @i) + REPLICATE('*', @i);
        PRINT @Line;
        SET @i = @i - 1;
    END
END

```

```

END

SET @i = 2;
WHILE (@i <= @Rows)
BEGIN
    SET @Line = SPACE(@Rows - @i) + REPLICATE('*', @i);
    PRINT @Line;
    SET @i = @i + 1;
END
END

ELSE IF @PatternType = 'Hollow Hourglass'
BEGIN
    -- Upper half
    SET @i = @Rows;
    WHILE @i >= 1
    BEGIN
        SET @Line = SPACE(@Rows - @i);
        SET @j = 1;

        WHILE @j <= (2 * @i - 1)
        BEGIN
            IF @j = 1 OR @j = (2 * @i - 1) OR @i = @Rows
                SET @Line = @Line + '*';
            ELSE
                SET @Line = @Line + ' ';
            SET @j = @j + 1;
        END

        PRINT @Line;
        SET @i = @i - 1;
    END
END

```

```

END

-- Lower half
SET @i = 2;
WHILE @i <= @Rows
BEGIN
    SET @Line = SPACE(@Rows - @i);
    SET @j = 1;
    WHILE @j <= (2 * @i - 1)
    BEGIN
        IF @j = 1 OR @j = (2 * @i - 1) OR @i = @Rows
            SET @Line = @Line + '*';
        ELSE
            SET @Line = @Line + ' ';
        SET @j = @j + 1;
    END
    PRINT @Line;
    SET @i = @i + 1;
END

END

ELSE IF @PatternType = 'Floyds Triangle'
BEGIN
    DECLARE @num INT = 1;    -- starting number
    WHILE @i <= @Rows
    BEGIN
        SET @Line = '';
        SET @j = 1;

        WHILE @j <= @i

```

```

        BEGIN
            SET @Line = @Line + CAST(@num AS VARCHAR(10)) + ' ';
            SET @num = @num + 1;
            SET @j = @j + 1;
        END

        PRINT @Line;
        SET @i = @i + 1;
    END
END

ELSE IF @PatternType = 'Pascals Triangle'
BEGIN
    DECLARE @k INT, @C INT;
    WHILE @i <= @Rows
    BEGIN
        SET @C = 1;
        SET @k = 1;
        SET @Line = SPACE(@Rows - @i);

        WHILE @k <= @i
        BEGIN
            SET @Line = @Line + CAST(@C AS VARCHAR(10)) + ' ';
            SET @C = @C * (@i - @k) / @k;
            SET @k = @k + 1;
        END

        PRINT @Line;
        SET @i = @i + 1;
    END
END

```



```

END

ELSE
BEGIN
    PRINT 'Invalid Pattern Type Selected';
END

```

----- Output

